

最大流增广路径推导：从正向加流到残量网络

这份笔记只记录顺着推导的过程，不把残量网络当作一开始就知道的答案。核心问题是：当前已经有一个合法 flow 时，如果又找到一条看起来可以增加的路径，怎样把这条路径叠加到原始 flow 上，并且不破坏容量限制和流量守恒？

最容易想错的地方是：以为增广路就是最终 flow

的真实走向。实际上最大流里的增广路更像一组有符号更新指令。它告诉我们哪些原边增加，哪些旧 flow 要撤销。

1. 从当前 flow 出发，而不是从算法名词出发

先不提 Ford-Fulkerson，不提残量网络。只假设我们已经有一个当前合法 flow。合法的意思是：每条边不超过容量；每个中间点的流入等于流出。这个 flow 未必最优，它只是当前可行解。

下一步自然要问：当前 flow 还能不能增加？这和二分图匹配里“当前 matching 能不能 +1”是同一类问题。不是一口气构造最优解，而是先找一个能改进当前解的证据。

2. 正向剩余容量算法为什么自然

当前 flow 给定后，原图每条边都有 $\text{remaining capacity} = \text{capacity} - \text{flow}$ 。如果有一条从 s 到 t 的路径，路径上每条边 remaining capacity 都大于 0，那么沿这条路径加一点 flow 看起来完全合理。

路径上最多能加多少？就是这条路径上最小的 remaining capacity，也就是瓶颈。加到瓶颈后，至少一条边被打满；容量约束仍成立。中间点因为同时多了一份流入和流出，所以守恒约束也成立。

3. 直接叠加什么时候确实没问题

如果增广路径完全由原图正向边组成，那么更新规则非常简单：路径上的每条边 $\text{flow} += \text{delta}$ 。比如 flow 已经是 4，找到正向路径 $s \rightarrow a \rightarrow b \rightarrow t$ ，瓶颈是 1，那么直接把这三条边各加 1，新的 flow 就是 5。

这就是为什么一开始会觉得“直接原图就可以”。这个判断在全正向路径场景下没有错。问题是：最大流的困难不在这个场景，而在旧 flow 占住了错误位置的时候。

4. 当前 flow 不是历史事实

current flow 不是不可改变的历史，它只是一个可行解。如果它阻塞了更大的流，我们必须允许它被改写。这个“允许改写”正是残量网络的来源。

类比二分图匹配：当前 matched edge 不是永远固定的。增广路翻转时，某些已有匹配边会被删掉，某些非匹配边会被加进来。最大流里也是同样的思想，只是数值是容量而不是 0/1 匹配。

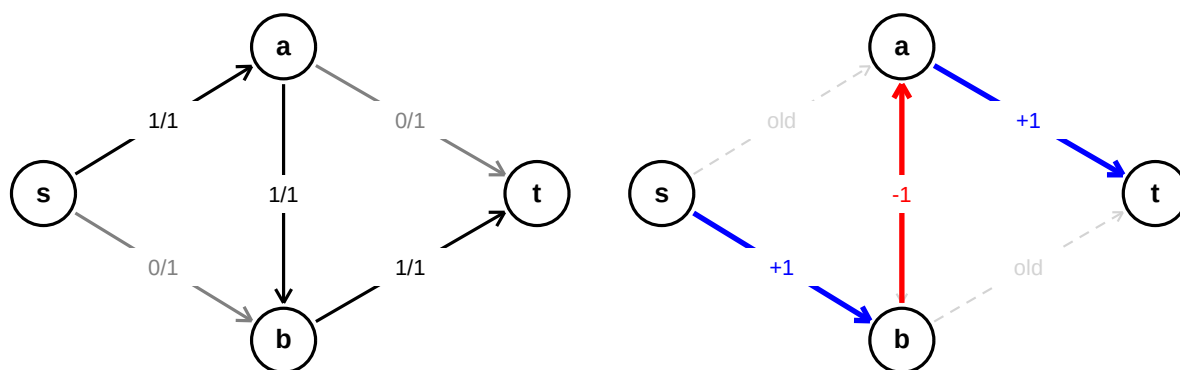
5. 最小反例：只看正向剩余会误停

考虑四个点 s, a, b, t，所有容量都是 1： $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow b$, $a \rightarrow t$, $b \rightarrow t$ 。当前 flow 先走了 $s \rightarrow a \rightarrow b \rightarrow t$ ，所以 $s \rightarrow a$ 、 $a \rightarrow b$ 、 $b \rightarrow t$ 都满了，总 flow 为 1。

从正向剩余容量看，s 还能走到 b，a 还能走到 t，但 b 没有正向边到 a， $b \rightarrow t$ 又满了。所以只看原图正向剩余边，会误以为找不到 s 到 t 的增广路。

当前 flow：只看正向剩余会卡住

调整路径：s -> b -> a -> t



正确的改法不是继续寻找原图正向路径，而是撤销 a->b 上的旧 flow，让 a 改走 a->t，同时让 s 走 s->b，b 保持到 t。也就是说，要把 s->a->b->t 这条旧流的一部分拆开，改成 s->a->t 和 s->b->t。

6. 反向边不是原图边，而是撤销旧 flow 的权限

a->b 当前有 1 单位 flow。为了取消它，需要一种方式表示“从 b 回到 a”。这并不是说原图真的能从 b 运输到 a，而是说可以把 a->b 上已经占用的 1 单位 flow 退回去。

因此，只要原边 u->v 上有 f 单位 flow，当前调整结构里就应该允许 v->u，容量为 f。这个容量表示最多能撤销多少旧 flow。

7. 增广路是有符号更新，不是最终流向

在反例里，调整路径是 s->b->a->t。看起来有一条 b->a 的边，但最终真实 flow 里没有 b->a。它只是一条反向调整边。

沿 s->b->a->t 增广 1 的实际含义是：s->b 加 1；a->b 减 1；a->t 加 1。最终 flow 仍只存在于原图边上。

8. 更新向量写清楚

有符号更新：增广路不是最终真实流向

s->b	+1	原图正向边增加
b->a	对 a->b 做 -1	撤销旧的 a->b flow
a->t	+1	原图正向边增加

最终 flow 仍只记录在原图边上，而不是记录在 b->a 这条反向调整边上。

9. 为什么守恒不被破坏

沿残量路径更新时，每个中间点在更新向量里仍是一进一出。只不过有些“一进一出”对应正向增加，有些对应撤销旧 flow。

以 a 为例，反向边 $b \rightarrow a$ 表示 $a \rightarrow b$ 减少 1，而 $a \rightarrow t$ 增加 1。 a 原本流出到 b 的 1 被换成流出到 t 的 1； a 的净守恒仍然成立。

10. 残量网络被自然推出

残量网络不是额外魔法，而是当前所有合法调整动作的集合。原边还有剩余容量，就有正向残量边；原边已有 flow，就有反向残量边。

在残量网络里找 s 到 t 的路径，等价于寻找一组可以让总 flow 增加的正负更新。

11. 正向算法知道什么，不知道什么

只看正向剩余容量的算法知道哪些边还可以直接加 flow；它不知道哪些旧 flow 可以撤销。

因此它不是完全错误，而是不完整：它只支持“继续加”，不支持“反悔再重排”。最大流必须支持后者。

12. 和二分图匹配的关系

二分图匹配转最大流后， $source \rightarrow left \rightarrow right \rightarrow sink$ 容量全为

1。未匹配边对应正向剩余边，已匹配边对应可以反向走的残量边。

所以匹配里的交替增广路，本质就是最大流残量增广路在这个特殊网络中的投影。

13. DFS、BFS、最大瓶颈路径只是选路策略

一旦残量网络被定义出来，算法就是反复找 s 到 t 的残量路径。DFS 可以找，BFS 可以找，也可以试图找瓶颈最大的路径。

这些策略改变迭代次数和复杂度分析，但不改变最大 flow 的最终值。

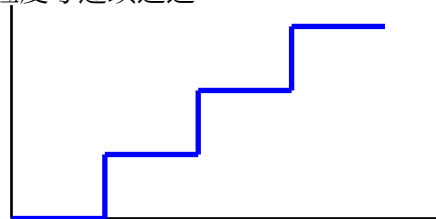
14. 整数、有理数和实数容量

整数容量下，每次按瓶颈增广至少增加 1

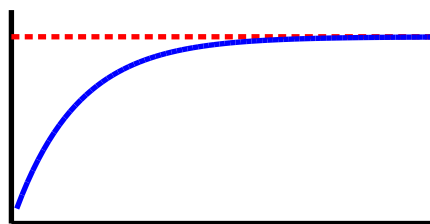
个单位，所以有限步终止。有理数容量可以统一乘公共分母，缩放成整数。

任意实数容量下，坏的选路可能产生有界单调递增序列，不断逼近最大流但不在有限步达到。这不是容量无限，而是没有最小粒度。

离散粒度与连续逼近



整数/有理数：有限步



实数：可能只逼近

15. 离散与连续的思维分界

离散算法通常依赖有限状态、最小步长和组合结构。连续优化更关心收敛性、误差、极限和稳定性。

最大流在整数容量下是典型离散算法；一旦允许任意实数容量，终止性问题就带上数学分析味道。

16. 最终框架

当前 flow 不是终点；残量网络表达当前 flow 的所有调整自由度；增广路是一条从 s 到 t 的有符号更新路径；瓶颈决定这次能更新多少；找不到残量路径时，当前 flow 才不能再通过这种方式改进。这条推导的关键不是背“反向边”三个字，而是理解：反向边是为了把旧 flow 从错误结构中释放出来。

17. 反例中的边状态表

边	当前状态	正向剩余 / 反向可撤销
s->a	1/1	正向剩余 0；可反向撤销 1
a->b	1/1	正向剩余 0；可反向撤销 1
b->t	1/1	正向剩余 0；可反向撤销 1
s->b	0/1	正向剩余 1
a->t	0/1	正向剩余 1

这张表说明：正向算法只会看到 s->b 和 a->t，但它不知道 a->b 可以被撤销，因此无法把 b 和 a 接起来。残量网络加入 b->a 后，s->b->a->t 才成为一条完整的调整路径。

18. 推导轨迹：从“直接加”到“允许反悔”

第一层直觉是最朴素的：当前 flow 既然已经合法，那每条边剩多少容量就很清楚。只要还能从 s 沿着剩余容量走到 t，就把这条路径加进去。这个想法不是错，而是只覆盖了“旧 flow 不需要改”的情况。

第二层卡点是：一个 flow 不是若干条永久固定的路径记录。即使我们曾经用 s->a->b->t 描述当前 flow，算法内部真正保存的是每条边上的数值。路径分解只是解释方式，不是状态本身。

第三层转向是：如果当前 flow 的某一段占错了位置，新的增广不一定是纯新增，它可能包含撤销。这个撤销不是删除整条路径，而是对某条原边的 flow 做减法。

第四层抽象是：既然“可新增”和“可撤销”都是合法调整，那我们应该把二者放到同一个搜索空间里。正向剩余边表达可新增，反向残量边表达可撤销。

第五层理解是：残量网络不是另外一个真实网络，而是当前 flow 周围的局部可行方向集合。它类似于优化问题里的“当前点附近还能往哪些方向移动”。

第六层才是算法：在这个调整空间里找一条从 s 到 t 的路。找到后，不是把这条路当成最终流向，而是把它解释成一个更新向量。

第七层是终止判断：如果连这个调整空间里都找不到 s 到 t 的路，说明所有由当前 flow 出发的合法增广方向都被封死了。这时才可以说当前 flow 已经最大。

19. 正向路径叠加和残量路径叠加的差别

情况	路径含义	更新规则
全正向路径	每条边都是原图可继续加路径上所有原边 flow += delta	
含反向边路径	部分边表示撤销旧 flow	正向边 += delta；反向边对应原边 -= delta
最终 flow	只在原图边上记录	不记录反向边，它只是调整指令

这张表是整个推导的核心。全正向路径是残量路径的特例；残量路径是更一般的更新向量。只理解第一行，就会得到“直接看原图正向剩余容量”的算法；理解第二行，才会自然得到残量网络。

20. 和二分图增广路的并列理解

二分图匹配	最大流
当前 matching	当前 flow
未匹配边可以加入	正向剩余边可以加流
已匹配边可以被翻转出去	已有 flow 可以被反向撤销
交替路径	残量路径
翻转后匹配数 +1	有符号更新后 flow 增加 delta

这也是为什么两个算法看起来“像又不像”。它们同属于增广框架，但二分图匹配由于容量全是 1、结构分左右两层，所以残量路径表现为非匹配边和匹配边交替。最大流更一般，边上的数值可以是任意容量，因此反向调整量也可以是任意 delta。

21. 为什么“增广路不是最终路径”这个区分重要

如果把增广路误认为最终流向，就会被 $b \rightarrow a$ 这样的反向边困住：原图明明没有 $b \rightarrow a$ ，为什么算法能走？正确解释是：算法不是让货物真的从 b 运到 a ，而是在原有 $a \rightarrow b$ 的 flow 上做减法。

最终 flow 可以重新分解成若干条 s 到 t

的真实路径，但这种分解可能和某次增广路径完全不同。一次增广操作是中间调整，不是最终解释。

这个区分对写代码也重要：残量图中走到反向边时，不能创建一条新的原图边；必须修改原边的 flow 值。也就是对 edge 和 reverse edge 的 flow/capacity 成对更新。

22. 关于选路策略的定位

DFS 找一条能走的残量路径，是最朴素的增广框架。BFS 找最短边数路径，是 Edmonds-Karp 的选择策略。Dinic 先 BFS 分层，再 DFS

在分层图里推多条流。它们的共同点都是在残量网络中找可增广结构。

你提到“如果有多条路径，选可充容量最大的那条”也是一种合法选路策略。在整数和有理数容量下，只要每次沿合法残量路径按瓶颈增广，最终最大 flow 值不会因选路而改变。选路影响的是轮数和证明复杂度。

因此，算法框架和选路策略要分开：残量网络定义的是问题还能怎么调整；DFS/BFS/最大瓶颈定义的是这次先调整哪一条。

23. 有理数、无理数和有限终止

整数容量的关键不是“数值简单”，而是存在离散最小单位。每次瓶颈增广至少增加 1，所以不可能无限逼近而不停。

有理数容量可以乘公共分母化成整数。比如所有容量都是分母不超过某个公共 D 的有理数，那么整体乘以 D 后，增广量也有统一粒度。

无理数容量的病态不在于容量无限大。sqrt(2)

这样的容量仍然有限。问题在于没有统一最小粒度，坏的选路可能让 flow 值形成有上界的单调递增序列，不断逼近最大值但有限步不取到。

这就是你说的“有界、逼近、就是达不到”的数学分析味道。它不是竞赛和面试的主线，但知道这个边界可以帮助理解为什么理论教材会强调容量类型。

24. 离散算法和连续优化的思维差异

在离散算法里，常见困难是可能性太多：路径、子集、排列、状态组合会爆炸。解决方法通常是找不变量、等价类、闭包、状态压缩。

在连续优化里，常见困难是过程不受控：是否收敛、收敛到哪里、误差多大、数值是否稳定、步长如何选。最大流的实数容量终止性问题正好把这两类困难接在一起。

图结构是离散的，但容量进入连续实数后，算法终止性突然变成极限问题。这就是这个话题有“哲学延伸”的地方。

25. 一句话压缩

最大流增广的本质不是“找一条新运输路线”，而是“找一条能改善当前 flow 的有符号调整路线”。正向边表示增加，反向边表示撤销。残量网络把这两种动作统一成普通图搜索。